

SQL Cheat Sheet for the AI Era

AI writes the first draft.
You still own the number.

Intent

Schema

Joins

NULLs

Aggregates

Windows

Performance

Privacy

Sagar Rathkanthiwar

Data & AI Professional

12 slides · save for later →

The query runs. That doesn't make it right.

90% use it • 30% don't trust it

Nine in ten developers now work with AI. Nearly a third report little to no trust in the code it hands back (DORA, State of AI-assisted Software Development).

The verification tax

DORA's finding: time saved writing gets spent auditing. AI raises the volume of code far faster than it raises your confidence in it.

Your schema is the hard part

On real enterprise warehouses rather than textbook databases, leading text-to-SQL systems still land well short of expert accuracy — undocumented columns and ambiguous join paths, not syntax.

The bottleneck moved from **writing** SQL to **checking** it. Everything after this slide is the checking part.

Split the work into three lanes

ASK AI

First drafts, dialect translation, boilerplate, alternate approaches, and plain-English explanations of SQL you inherited.

INSPECT YOURSELF

Grain, keys, join cardinality, filter logic, date boundaries, and what the business actually means by "active customer."

CATCH BEFORE SHIPPING

Duplicated rows, NULL logic, averages at the wrong grain, unbounded scans, and columns you shouldn't be reading.

AI is fast at SQL. You are accountable for the definition of **customer**, **active**, and **revenue**.

Prompt the schema, not just the question

Weak prompt "Give me total revenue by customer."

Tables + columns (with types):

orders(order_id PK, customer_id, order_ts, status, amount_usd)

refunds(refund_id PK, order_id FK, amount_usd)

Grain I want: one row per customer per month

Revenue means: amount_usd minus refunds, excluding status='test'

Dialect: Snowflake. Window: last 12 full months.

Write the query, then list every assumption you made.

Catch: an answer with no stated assumptions is an answer you have to reverse-engineer later. Make "list your assumptions" part of every prompt.

Know the grain before you trust the join

Grain = what one row means. Most wrong numbers are grain mistakes, not syntax mistakes — and the model can't see your data, only its names.

```
-- 1. Is this column actually a key?
SELECT COUNT(*) AS rows_total,
       COUNT(DISTINCT order_id) AS distinct_ids
FROM orders;                                -- the two must match

-- 2. Will this child table fan out my rows?
SELECT order_id, COUNT(*) AS n
FROM refunds
GROUP BY order_id
HAVING COUNT(*) > 1
LIMIT 20;                                   -- any result means one-to-many
```

Ask AI: "What grain does this query return, and which column makes it unique?" Then prove it with the two queries above.

Duplicates don't announce themselves

A one-to-many join silently multiplies rows — and every **SUM** that comes after it.

```
-- WRONG: each order is counted once per refund row
SELECT o.customer_id, SUM(o.amount_usd) AS revenue
FROM orders o
JOIN refunds r ON r.order_id = o.order_id
GROUP BY 1;

-- RIGHT: collapse the child table to the join grain first
SELECT o.customer_id,
       SUM(o.amount_usd) - SUM(COALESCE(r.refunded,0)) AS net_rev
FROM orders o
LEFT JOIN (SELECT order_id, SUM(amount_usd) AS refunded
           FROM refunds GROUP BY order_id) r
  ON r.order_id = o.order_id
GROUP BY 1;
```

Guard: row count before the join should equal row count after. If it grew, you owe yourself an explanation — not a `DISTINCT`.

NULL isn't a value. It's "unknown."

- `x = NULL` **is never true** — use `IS NULL`. Comparisons against unknown return unknown, not false.
- `NOT IN (subquery)` **returns zero rows** if that subquery contains a single NULL. Use `NOT EXISTS` instead.
- `COUNT(col)` **skips NULLs**, `COUNT(*)` doesn't. `AVG(col)` divides by the non-NULL count only.
- **A LEFT JOIN plus a WHERE filter on the right table** quietly becomes an INNER JOIN. Move that filter into the `ON` clause.

```
-- Safe anti-join: customers with no orders
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o
                  WHERE o.customer_id = c.customer_id);
```

Catch: ask AI "what happens here if this column is NULL?" — then test it with a row that actually is.

Check the math, not just the output

- **WHERE filters rows, HAVING filters groups.** Moving a row filter into HAVING changes the answer.
- **An average of averages is not an average.** Re-aggregate from base rows, weighted properly.
- **COUNT(DISTINCT user_id) is a different question** than COUNT(*). Decide which one you were asked.
- **Empty groups vanish.** If "0 orders in July" must appear, join a date spine.

```
SELECT d.month,  
       COUNT(DISTINCT o.customer_id) AS buyers,  
       COALESCE(SUM(o.amount_usd), 0) AS revenue  
FROM date_spine d  
LEFT JOIN orders o  
      ON DATE_TRUNC('month', o.order_ts) = d.month  
      AND o.status <> 'test'    -- filter goes in ON, not WHERE  
GROUP BY 1  
ORDER BY 1;
```

Catch: if a total looks too round or too small, check the filters before you check the data. Most "missing revenue" is a WHERE clause.

Rank, dedupe and compare without collapsing rows

Window functions keep every row and add context. GROUP BY collapses them. Know which one you actually asked for.

```
-- Latest order per customer (deterministic dedupe)
SELECT * FROM (
  SELECT o.*,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                             ORDER BY order_ts DESC, order_id DESC) AS rn
  FROM orders o
) ranked
WHERE rn = 1;

-- Month over month change
SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS mom_delta
FROM monthly_revenue;
```

Catch: ties. Without a tiebreaker in ORDER BY, "latest" can change between runs. And RANK vs DENSE_RANK vs ROW_NUMBER is a business decision, not a style choice.

Correct and expensive is still a problem

- **Read the plan, not the vibes.** `EXPLAIN` / `EXPLAIN ANALYZE`, and check bytes scanned on cloud warehouses where you pay per scan.
- **Keep predicates sargable.** `WHERE order_ts >= '2026-01-01'` can use an index or partition; `WHERE YEAR(order_ts) = 2026` usually can't.
- **Filter partition and cluster keys early**, and stop selecting `*` from tables that run to hundreds of columns.
- **Treat a stray `DISTINCT` as a smell.** It's often patching a fan-out join instead of fixing it.

Ask AI: "Rewrite this to scan fewer partitions and explain the tradeoff." Then read the plan yourself — cost estimates are the one thing the model genuinely cannot see.

Review what the query exposes

- **Never concatenate user input into SQL.** Parameterize. A generated f-string that "works" is an injection waiting for its first quote character.
- **Least privilege by default.** Analysis runs on a read-only role. Write credentials don't belong in a notebook or an agent's environment.
- **Don't paste real PII or secrets into a prompt** to "help it understand the data." Share column names, types and synthetic rows instead.
- **Query only what you're entitled to** — prefer masked or governed views, and re-check access before sharing an export.
- **No UPDATE or DELETE** until you've run the identical predicate as a SELECT, inside a transaction you can roll back.

Catch: generated SQL inherits whatever access you hand it. The model has no idea which of your columns are regulated — that judgement is entirely yours.

Six checks before you trust the number

- 1** **Row counts** are sane before and after every join.
- 2** **Grain matches the ask** — one row per ____.
- 3** **Three records traced end-to-end** back to the source system.
- 4** **Totals reconcile** with a known report or last month's figure.
- 5** **Edge cases handled**: no orders, refunds only, NULLs, future dates.
- 6** **Re-run is stable** — same input, same output, no random ties.

Save this, then run check #1 on the query you shipped yesterday.

Follow [Sagar Rathkanthiwar](#) for practical Data & AI

Sources: DORA, State of AI-assisted Software Development & ROI of AI-assisted Software Development (dora.dev) · Spider 2.0 enterprise text-to-SQL benchmark and leaderboards (spider2-sql.github.io, arXiv 2411.07763). SQL behaviour described here is standard-SQL; syntax details vary by dialect.